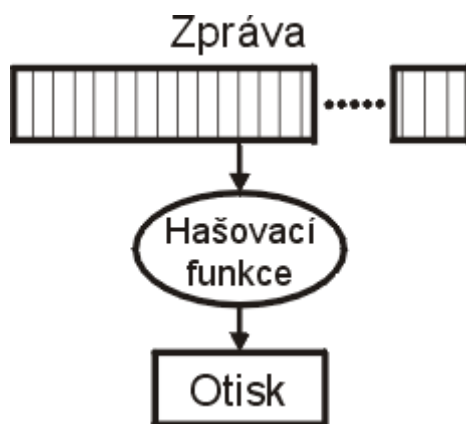


Úvod do hašovacích funkcí

Hašovací funkce je funkce h , která má přinejmenším tyto vlastnosti: je kompresní - provádí mapování argumentu /vstupu/ x libovolné bitové délky na hodnotu $h(x)$ /výstup/, která má pevně určenou bitovou délku, je snadno vypočitatelná – pro dané h a argument x je snadné vypočítat $h(x)$.

Tuto vlastnost ilustruje obrázek č.1, na němž je argument označen jako „zpráva“ a hodnota funkce slovem „otisk“. Smyslem použití hašovacích funkcí obecně bývá možnost reprezentovat potenciálně i velmi dlouhou zprávu jen jejím krátkým otiskem a vytvořit pomyslnou relaci 1:1 mezi zprávou a otiskem.



Kompresní vlastnost všech hašovacích funkcí

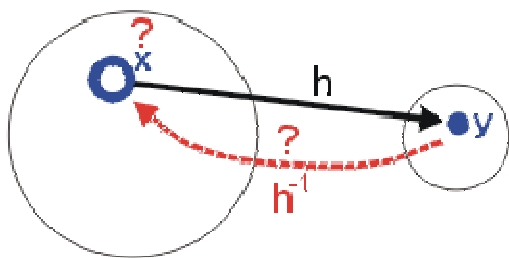
Hašovací funkce mají v počítačové praxi velmi mnoho použití. V diagnostice hardware např. indikují konečné stavy testů, v komunikaci se používají pro detekci chyb (kódy CRC apod.), v softwarových technikách např. pro implementaci tabulek atd. Ve všech těchto případech se zpravidla vyžaduje, aby hašovací funkce prováděla i dobrou náhodnou distribuci výsledků, rovnoměrné pokrytí oboru hodnot, aby malá změna vstupního řetězce vedla jistě k odlišnému výsledku (ideálně lavinově jinému).

Hašovací funkce vyhovující výše uvedeným aplikacím sice dobře detekují „náhodné“ změny argumentů (zprávy), nejsou však dostatečné pro obor kryptologie, ve kterém lze předpokládat, že útočník bude postupovat při změnách argumentů funkce s veškerou možnou inteligencí.

Kryptologové se proto, mimo jiné, zabývají hašovacími funkcemi (funkci značíme h , argumenty x a x' /vstupy/, hodnoty y a y' /výstupy/), které mají následné tři vlastnosti:

Odolnost argumentu (preimage resistance) – pro v podstatě všechny výstupy je výpočtově neschůdné nalézt jakýkoliv vstup, který hašuje na daný výstup; tj. nalézt jakýkoliv argument x takový, že $h(x) = y$ pro danou hodnotu y , pro niž odpovídající vstup není předem znám.

Pro ne-matematiky ilustruje tuto vlastnost další obrázek..



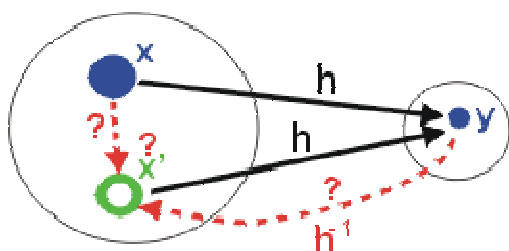
Odolnost argumentu (též „jednocestnost“)

Tato vlastnost polopaticky řečeno znamená, že z otisku nesmíme být schopni zjistit jakoukoliv zprávu, které by odpovídal.

Předem známa je pouze funkce h , tj. způsob jejího výpočtu, definiční obor (kruh vlevo) i obor hodnot (kruh vpravo) této funkce. Z oboru hodnot je volen libovolný výstup y . Nalezení jakéhokoliv vstupu x funkce h , který by mu odpovídal, nesmí být možné. Definice uvedená výše připouští, že útočník může předpočítat obraz (hodnoty y) pro nějakou zanedbatelně malou podmnožinu vzorů z definičního oboru – takové způsoby nalezení argumentu se neuvažují, protože se předpokládá, že se útočník na y netrefí.

V praxi je v takové situaci například útočník na heslo x , které je uloženo formou zhašované hodnoty y v souboru na disku, k němuž se mu podařilo získat přístup (pro ztížení slovníkových útoků bývají hodnoty hesla ještě tzv. osoleny, ale to zde rozebírat nebudeme).

Odolnost druhého argumentu (2nd-preimage resistance) – je výpočtově neschůdné nalézt jakýkoliv druhý vstup, který má shodný výstup jako jakýkoliv určený první vstup; tj. pro dané x nelze nalézt druhý argument $x' \neq x$, přičemž platí $h(x) = h(x') = y$.

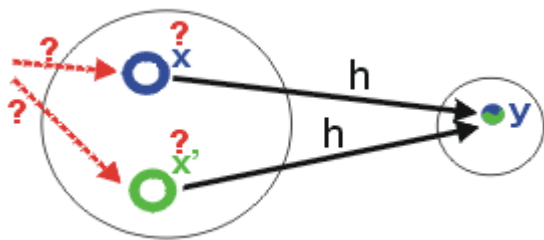


Odolnost druhého argumentu (též „slabá odolnost proti kolizím“)

Tato vlastnost se projeví v „normální“ situaci, kterou si většina uživatelů představí při útoku na hašovací funkci. Máme předem danu hašovací funkci h , nějakou zprávu x a její otisk y (nebo ho snadno spočteme). Nesmí být možné nalézt jakoukoliv jinou zprávu x' , jejíž otisk by byl stejný. Protože se předem neví, jakou zprávu budeme hašovat, výše uvedená definice požaduje tuto vlastnost pro všechny argumenty x .

V praxi přichází do úvahy útok tehdy, pokud by chtěl někdo zaměnit zprávu x za x' , což může přicházet do úvahy např. při (elektronickém) digitálním podpisu, při kontrole integrity souborů (ať již stahovaných z webu nebo P2P sítí). Běžný člověk by si pravil, že to by mohlo stačit, kryptologové však vznášejí ještě třetí požadavek.

Odolnost proti kolizím (collision resistance) – je výpočtově neschůdné nalézt jakékoliv dva rozdílné vstupy x a x' , které hašují na shodný výstup; tj. $x' \neq x$ takové, že $h(x) = h(x') = y$ (volný výběr obou vstupů).



Odolnost proti kolizím (též „silná odolnost proti kolizím“)

Tato situace se liší od předchozí tím, že si je možné hodnotu x volit libovolně. Na první pohled se zdá, že se jedná o shodné definiční podmínky jako v předchozím případě, ale rozhodně tomu tak není. Danost zprávy y (prvního argumentu) je totiž při zkoušení odolnosti druhého argumentu určena nějakou, na útočníkovi nezávislou, podmínkou. Z pohledu útočníka je y již dáno a musí pro něj hledat x' .

Při obecné (silné) odolnosti proti kolizím si však útočník může volit x i x' s jediným cílem, totiž aby hašovaly na shodnou hodnotu (lhostejno jakou). Tato vlastnost předpokládá, že útočník je v takové pozici, že může ovlivnit vstupní hodnotu hašovací funkce. Další viz níže u útoku Wangové, které se všechny zaměřují na toto obecné hledání kolizí (dle obr.4).

Ideální kryptografická hašovací funkce bude oplývat všemi třemi vlastnostmi! Je vhodné chápat, že pokud je velikost množiny definičního oboru hašovací funkce větší než oboru hodnot (což bývá v praxi vždy), hašovací funkce nemůže být bezkolizní.

Pozorný čtenář si však jistě všiml, že definice obsahují obraty „výpočtově neschůdné“ - kolize prostě nesmí být možné nalézt. Kontext definice se mění jak rozvojem technologie, tak požadovanou mírou ekonomických nákladů (nemá smysl utrácet více, než získáte).

Narozeninový paradox

Zajímavé rovněž je, že obě zmíněné situace (odolnost druhého argumentu a odolnost proti kolizím) mají rozdílný výpočet pravděpodobností náhodného nalezení kolizí. Druhá situace je v kryptologii známa jako tzv. narozeninový paradox.

Jste-li na večírku, pak pro 100% pravděpodobnost toho, že se na něm nachází někdo, kdo se narodil shodný den v roce (léta se neuvažují) jako právě vy, je zapotřebí, aby tam přišlo alespoň 366 osob (musíte jít na megapárty). Pokud se však spokojíte s 50procentní pravděpodobností toho, že se na večírku nachází libovolné dvě osoby se shodným narozeninami, pak dostačuje, aby večírek navštívilo 23 osob.

Pozn: narozeninový paradox samozřejmě předpokládá náhodnost narozenin dotyčných, tj. že nejste třeba na vítání občánků, ani nejste dvojčata, chodící spolu kazit statistiky. Uvedené pravděpodobnosti též zanedbávají přestupné roky.

Tato nevinná matematická hříčka má v kryptologii vážný dopad. Útoky hrubou silou jsou v případech použitelnosti narozeninového paradoxu totiž mnohem snazší a říká se jim pak narozeninový útok.

Message-Digest algorithm

Message-Digest algorithm je rozšířená rodina hašovacích funkcí, která vytváří ze vstupních dat výstup (otisk) fixní délky. Otisk je též označován jako miniatura, kontrolní součet (v zásadě nesprávné označení), fingerprint, hash (česky někdy psán i jako haš).

Jeho hlavní vlastností je, že malá změna na vstupu vede k velké změně na výstupu, tj. k vytvoření zásadně odlišného otisku.

MD5

Algoritmus MD5 se prosadil do mnoha aplikací (např. pro kontrolu integrity souborů nebo ukládání hesel). MD5 je popsán v internetovém standardu RFC 1321 a vytváří otisk o velikosti 128 bitů. Byl vytvořen v roce 1991 Ronaldem Rivestem, aby nahradil dřívější hašovací funkci MD4.

Bezpečnost

V roce 1996 byla objevena vada v návrhu MD5, a i když nebyla zásadní, kryptologové začali raději doporučovat jiné algoritmy, jako je například SHA (i když ani ten již dnes není považován za bezchybný). V roce 2004 byly nalezeny daleko větší chyby a od použití MD5 v bezpečnostních aplikacích se upouští.

Příklad kontrolního součtu MD5

Otisk 43 bajtového znakového řetězce (vyjádřený v hexadecimálním zápisu):

MD5("The quick brown fox jumps over the lazy dog") =

9e107d9d372bb6826bd81d3542a419d6

Stačí malá změna vstupního řetězce, aby byl otisk úplně odlišný (např. změňme d na c):

MD5("The quick brown fox jumps over the lazy cog") =

1055d3e698d289f2af8663725127bd4b

Zvýšení bezpečnosti

MD5 se často používá pro ukládání hesel. Přidáním soli k heslu se ztěžují útoky na získání hesla a účinnost slovníkových útoků s využitím předpočítaných tabulek (anglicky rainbow table attack). Tento postup lze použít i pro jakékoli kryptografické

hašovací funkce, které lze dopředu "předpočítat", jejich výsledný haš uložit a pak rychleji vyhledat v databázi srovnáváním s hašem, na který se útočí.

hash = MD5 (heslo . sul)

Pro jisté zvýšení bezpečnosti je možné kombinovat například heslo a uživatelské jméno, v takovém případě pokud dva uživatelé použijí totožné heslo, otisk (haš) jejich hesel bude zásadně odlišný, protože jejich uživatelská jména se určitě budou lišit. Další možností částečného zvýšení bezpečnosti je použití více hašovacích algoritmů najednou, například kombinace MD5 a SHA. Postup zajistí vyšší odolnost chráněné informace v případě, že bude při jedné z funkcí nalezena kolize. Například:

SHA1(MD5("login").MD5("heslo"))

Secure Hash Algorithm

SHA navrhla americká NSA a jako standard ho vydal NIST (Národní institut pro standardy v USA) jako americký federální standard. SHA je rodina pěti algoritmů: SHA-1, SHA-224, SHA-256, SHA-384 a SHA-512. Poslední čtyři varianty se souhrnně uvádějí jako SHA-2. SHA-1 vytvoří obraz zprávy dlouhý 160 bitů. Čísla u ostatních čtyř algoritmů značí délku výstupního otisku v bitech.

SHA-1 rozsekává vstupní zprávu na bloky o délce 512 bitů, poslední blok zprávy doplňuje a zarovnává, včetně přidání údaje o délce zprávy, na něž je vyhrazeno posledních 64 bitů. SHA-1 tak může zpracovávat vstupní zprávy o délce až do $2^{64}-1$ bitů, tj. cca 2.305.840 TB. Uvedením délky se výrazně ztěžuje možnost nalezení a výskytu kolizí mezi zprávami různých délek – kolize je primárně potřeba hledat mezi zprávami zcela shodné délky. Výstup SHA-1 má délku 160 bitů, tj. 20 bytů.

SHA-1 je, podobně jako jiné hašovací funkce, tzv. iterativní hašovací funkce. To znamená, že při zpracování každého 512bitového bloku se vždy použije stejný modul tzv. vnitřní kompresní funkce.

Kompresní funkce má dva vstupy, 160bitový (v prvním kroku je inicializován předepsaným inicializačním vektorem, v dalších krocích přenosy z výstupů minulých kroků jako tzv. kontext) a 512bitový (blok dat). Po zpracování posledního bloku se výstup kompresní funkce použije jako výstupní hodnota.

Použití

SHA se používá v mnoha různých protokolech či aplikacích, včetně TLS a SSL, PGP, SSH, S/MIME a IPsec, ale je využívána i pro kontrolu integrity souborů nebo ukládání hesel. SHA je považována za nástupce hašovací funkce MD5.

Útoky na MD5, SHA-1 a SHA-2

Pro prolomení prvního kritéria (tedy nalezení zprávy, která koresponduje se svým otiskem), může být použita hrubá síla při provedení 2^L výpočtů, kde L je velikost bitů v otisku zprávy. Toto se nazývá vzorový útok (preimage attack).

Útok na druhé kritérium (nalezení dvou rozdílných zpráv, které mají stejný otisk), které nazýváme kolize, vyžaduje jen $2^{L/2}$ výpočtů k provedení tzv. narozeninového útoku (birthday attack).

Útok na MD5 hash byl proveden čínským týmem profesorky Xiaoyun Wangové, který se jím v roce 2004 proslavil. Svou schopnost útoku na MD5 (a na další hašovací funkce) prokázal zveřejněním výsledných kolizí. Na počítači IBM P690 (32ti procesorový stroj) má nalezení kolize pro MD5 trvat asi hodinu. Jelikož si (alespoň někteří) kryptologové v rámci akademické spolupráce rádi navzájem dávají hádanky, Číňané podstatné podrobnosti své metody nezveřejnili. Ostatní kryptologové se následně během podzimu 2004 pokoušeli přijít ze zveřejněných dat a informací na to, jak to bylo provedeno.

Předního českého kryptologa Vlastimila Klímu pohltilo téma nadmíru, takže mu věnoval tři měsíce. Dne 5. března 2005 pak mezinárodně publikoval článek s provokativním názvem "Nalézání kolizí v MD5 – hračka pro notebook". Patrně ale nepřišel přesně na metodu Wangové, ale našel svou, která je celkově asi 4-6x rychlejší. Den poté, tj. 6. března, nechala prof. Wangová zveřejnit články detailně popisující prolomení MD5 a dalších funkcí. Tím se opět potvrdilo, že nejlepším způsobem uvolnění embarga na konkrétní informace je vytvoření vlastní náhrady.

Prof. Wangová také v únoru 2005 popsala útok na SHA-1 v řádu 2^{63} pokusů. Poté následovaly další práce, poslední byla prezentována na konferenci Eurocrypt 2009 od autorů Camerona McDonalda, Philipa Hawkese a Josefa Pieprzyka a ukazuje kolizi se složitostí v řádu 2^{52} výpočtů.

Tiger hash

Tiger je hašovací funkce, kterou v roce 1995 navrhli Ross Anderson a Eli Biham. Tato funkce produkuje kontrolní součet, neboli hash o délce 192 bitů, popř. 128 či 160 u verzí Tiger/128 a Tiger/160 (u těchto verzí se hash získává zkrácením z původní délky 192 bitů). Používá se pro kontrolu integrity souborů nebo ukládání hesel.

Tiger2 je varianta funkce Tiger, která používá stejné zakončení vstupních dat, jako funkce MD5 či SHA-1, oproti mírně odlišnému zakončení dat v původní funkci Tiger. Oficiální specifikace funkce Tiger2 dosud nebyla publikována.

Tiger se často používá v tzv. Merklově hashovém stromě, kde je označován jako TTH (Tiger Tree Hash). TTH se používá například v mnoha P2P aplikacích, např. Direct Connect nebo Gnutella.

Otisk 43bitového znakového řetězce (vyjádřený v hexadecimálním zápisu):

```
Tiger("The quick brown fox jumps over the lazy dog") =  
6d12a41e72e644f017b6f0e2f7b44c6285f06dd5d2c5b075
```

```
Tiger2("The quick brown fox jumps over the lazy dog") =  
976abff8062a2e9dcea3a1ace966ed9c19cb85558b4976d8
```

Stačí malá změna vstupního řetězce, aby byl otisk úplně odlišný (např. změňme d na c):

```
Tiger("The quick brown fox jumps over the lazy cog") =  
a8f04b0f7201a0d728101c9d26525b31764a3493fcd8458f
```

```
Tiger2("The quick brown fox jumps over the lazy cog") =  
09c11330283a27efb51930aa7dc1ec624ff738a8d9bdd3df
```

Nulový vstupní řetězec produkuje následující kontrolní součet:

```
Tiger("") =  
3293ac630c13f0245f92bbb1766e16167a4e58492dde73f3
```

```
Tiger2("") =  
4441be75f6018773c206c22745374b924aa8313fef919f41
```

Budoucnost

V současnosti probíhají snahy o vývoj vylepšených hešovacích funkcí. NIST usiluje o zavedení jednoho nebo více dalších hešovacích algoritmů pomocí veřejné soutěže stejně jako u hledání Advanced Encryption Standard (AES) - první návrhy na SHA-3 byly podávány do 31. října 2008 a vyhlášení vítěze a publikace nového standardu je naplánováno na rok 2012.

Podle oficiální zprávy z 24. 7. 2009 NIST vybral do 2. kola soutěže o nový hašovací standard SHA-3 14 kandidátů. Překvapením je, že nepostoupil EDON-R, nejrychlejší z kandidátů. Zklamáním je jistě i to, že nepostoupil MD6, několik let vyvíjený a snad nejvíce teoreticky rozpracovaný algoritmus, za nímž stál největší vývojový akademický tým. NIST také vybral 14 místo 15 kandidátů a udělal to dokonce o měsíc dříve, než slíbil. Algoritmy, které postoupily, naleznete v následující tabulce.

Algoritmus	64bit	32bit	Autorský tým, poznámka
BMW	7/3	7/12	Mezinárodní tým 6 lidí, Gligoroski, Knapskog, El-Hadedy, Amundsen, Mjølhusnes (Norw. Univ.), Klima
Shabal	8	10	Francouzský tým 14 lidí (DCSSI, EADS, Fr. Telecom, Gemalto, INRIA, Cryptolog, Sagem)
BLAKE	8/9	9/12	Mezinárodní tým 4 lidí, Aumasson, Henzen, Meier, Phan (Switzerland, UK)
SIMD	11/12	12/13	Francouzský tým 3 lidí, Leurent, Bouillaguet, Fouque
Skein	7/6	21/20	Mezinárodní tým 8 lidí, Schneier, Ferguson, Lucks, Whiting, Bellare, Kohno, Callas, Walker
CubeHash	160/160 13/13	200/200 13/13	Dan Bernstein, (Univ. of Illinois), v 2. řádce rychlost uvažovaného tweaku
SHA-2	20/13	20/40	NIST, stávající standard (nesoutěží, pouze pro srovnání)
JH	16	21	Hongjun Wu, Inst. for Inf. Res., Singapore
Luffa	13/23	13/25	Mezinárodní tým 3 lidí, Canniere (Kath. Univ. Leuven), Sato, Watanabe (Hitachi)
Hamsi	25	36	Özgül Küçük (Kath. Univ. Leuven)
Grøstl	22/30	23/36	Mezinárodní tým 7 lidí, Gauravaram, Mendel, Knudsen, Matusiewicz, Rechberger, Schlaeffer, Thomsen
SHAvite-3	26/38 18/28	35/55 26/35	Izraelský tým (Dunkelman, Biham), s Intel AES instrukcemi 8 cyklů/bajt, Bernsteinova měření viz 2. ř.
Keccak	10/20	31/62	Mezinárodní tým 4 lidí (Bertoni, Daemen, Peeters, Van Assche, STM, NXP)
Echo	28/53	32/61	Mezinárodní tým 7 lidí (Billet, Gilbert, Rat, Peyrin, Robshaw, Seurin), Intel AES instr. ho urychlí
Fugue	28/56	36/72	Americký tým 3 lidí Halevi, Hall, Jutla (IBM)

Tabulka: Rychlost kandidátů v cyklech na bajt pro 64/32bitové procesory (1./2. sloupec) a pro 256/512 bitové varianty hašovacích funkcí (v buňce tabulky)

Do závěrečného kola soutěže postoupily algoritmy BLAKE, Grøstl, JH, Keccak a Skein. NIST výběr zdůvodnil rychlostí sw i hw implementace, složitostí hw řešení,

možností útoků na algoritmus nebo odlišností algoritmu od ostatních. Na druhou stranu je zarážející, že jeden nejrychlejších algoritmů BMW byl vyřazen právě kvůli své rychlosti se zdůvodněním: „too fast to be true“.

Vítěz byl NISTem vyhlášen 2. října 2012 a stal se jím algoritmus Keccak (kečék).

Keccak vytvořili Michaël Peeters z NXP Semiconductors, Guido Bertoni, Gilles Van Assche a Joan Daemen ze společnosti STMicroelectronics. Algoritmus umožňuje generovat volitelně dlouhý otisk, je možné použít 28, 32, 48 nebo 64 bajtů.



The Keccak Team. From left to right: Michaël Peeters, Guido Bertoni, Gilles Van Assche and Joan Daemen.

Ostatní finalisty soutěže stejně jako starší SHA-2 překonal Keccak v rychlosti při implementaci v hardware. Při běhu na běžném procesoru Core 2 má rychlost zhruba 13 cyklů na bajt. Jeho další výhodou je princip zcela odlišný od SHA-2, což znamená, že průlomový pokrok, který by ohrozil bezpečnost jedné z funkcí, pravděpodobně neohrozí druhou z nich.

Rozhodnutí standardizovat Keccak jako SHA-3 přineslo dost velké rozčarování nad stavem rozhodovacích procesů v NIST. Především díky faktu, že byl vybrán kandidát, který neodpovídal zadání. Kryptolog Bruce Schneier se vyjádřil v tom smyslu, že by bylo lepší, kdyby NIST nevybral žádného kandidáta. Vlastimil Klíma to shrnul takto (viz Cryptoworld 11–12/2012):

NIST požadoval, aby nový tank byl rychlejší i bezpečnější než stávající. [...] Po výběru vítěze vydal NIST závěrečnou zprávu, kde svůj výběr zdůvodnil. Z ní se dozvídáme, že Keccak není ani nejrychlejší, ani nejmenší (vyjádřeno „plochou křemíku“), ale má nejlepší poměr rychlosti k ploše křemíku. U našeho průměru s

tankem by soutěž vyhrál tank, který není ani nejrychlejší, ani nejlevnější, ale který má nejlepší poměr rychlosti ku hmotnosti.

Kryptolog Nicolas Curtois řekl, že celá SHA-3 soutěž přinesla jen vyhozené peníze a že výběr algoritmu Keccak leda tak zajistí kryptoanalytikům další práci – bude o čem psát publikace. Podobně se Nicolas vyjádřil i k otázce, proč se víc nevyužívají primitiva, která jsou v určitém modelu prokazatelně bezpečná, nebo je za nimi alespoň nějaký náznak důkazu. Třeba algoritmy RSA, DSA a ECDSA jsou založeny na těžkých matematických problémech; o symetrických šifrách to až na výjimky říct nelze.

Za výběrem šifer je podle Nicolase ze značné části politika. Dokonce se prý roznáší (snad pod tlakem NSA?), že „stream ciphers are not sexy“. Tudíž opět příliš mnoho pochyb a politiky na úkor skutečné kryptografie.

Zdroje:

<http://www.lupa.cz/clanky/hasovaci-funkce-jak-se-odolava-hackerum/>
<http://www.lupa.cz/clanky/zustava-elektronicky-podpis-bezpecny/>
http://en.wikipedia.org/wiki/SHA_hash_functions
http://cs.wikipedia.org/wiki/Narozeninový_problém
http://cs.wikipedia.org/wiki/Secure_Hash_Algorithm
http://cs.wikipedia.org/wiki/Tiger_%28hash%29
http://cryptography.hyperlink.cz/2009/crypto78_09_2nd_round_vk.pdf
<http://it.slashdot.org/story/10/12/11/0028226/SHA-3-Finalist-Candidates-Known?from=rss>
<http://cs.wikipedia.org/wiki/SHA-3>
<http://keccak.noekeon.org/>